Guía esencial de C# para la defensa (2 h 30)

Oct 14, 2026 · @juan carlos bodoque III

1. Cómo usar esta guía en 2 h 30

Esta es la versión comprimida de la guía completa de C#: conserva solo lo que más probablemente te preguntarán y lo que necesitas para explicar el código sin leerlo. Tiene unas 13.000 palabras en lugar de 49.000 y mantiene el banco de preguntas, con respuestas cortas. La lectura son unos 75 minutos; el resto del tiempo es para **practicar en voz alta**, que es lo que realmente prepara para la defensa.

Bloque	Secciones	Lectura	Práctica	Al terminar deberías poder…
1. Mapa y C# imprescindible	2 y 3	9 min	6 min	Explicar qué es una clase `static`, una propiedad, un evento y `using`
2. SQL dentro de C#	4	9 min	10 min	Reescribir `YaExisteDirector` de memoria y explicarla línea por línea
3. Conexión, errores y transacciones	5	7 min	8 min	Contar el viaje de un error desde `THROW` hasta el aviso en pantalla
4. Pantallas	6	6 min	6 min	Explicar `cargando`, `hayError` y el CRUD de Personal
5. Seguridad	7	7 min	8 min	Contar el login, BCrypt, el bloqueo y la recuperación con sus límites
6. Diseño por código	8	6 min	6 min	Explicar `Invalidate`, el dibujo de un icono y el redondeo
7. Flujos de punta a punta	9	5 min	7 min	Contar el login, el préstamo y la planilla con la receta de cinco frases
8. Preguntas	10	22 min	8 min	Responder cada pregunta con tus palabras, sin mirar
9. Repaso final	11	3 min	17 min	Decir el glosario y la lista de defectos conocidos

Suma 150 minutos justos: aprovecha los descansos entre bloques y, si vas sobrado, usa el tiempo extra en las preguntas marcadas con ★.

Qué se recortó. Los exportadores a Excel y PDF, la construcción de los diálogos, el detalle del dibujo de cada icono, los hilos y `async`, LINQ, y las preguntas menos probables (las 73 de la guía completa quedaron condensadas en 40, fusionando las que se parecen). Si te preguntan algo de eso, está en la guía completa; con lo que queda aquí puedes defender el proyecto.

Cómo estudiar. Lee una sección, ciérrala y cuenta en voz alta lo que entendiste como si hablaras con el profesor. Si te trabas en un punto, reléelo solo a él. No memorices números de error ni colores; entiende **por qué** se hace cada cosa, que es lo que se pregunta.

2. El mapa del código

La idea que debes llevar a la defensa: **el C# no guarda nada ni decide las reglas de fondo; pide, valida lo cómodo, pinta y reacciona. Guardar y garantizar lo hace SQL Server.** Si dominas tres cosas, dominas casi todo: (1) cómo se escribe y se manda una consulta SQL desde C#, (2) cómo se organiza una pantalla (carga, filtro, validación, guardado) y (3) cómo se dibujan los controles propios.

Pieza	Dato
Proyectos	2: `Modelos` (datos, servicios, exportadores) y `Vistass` (la aplicación con las pantallas). .NET Framework 4.8
Código escrito a mano	unas 17.600 líneas en 65 archivos
Código de diseñador	unas 14.400 líneas en 30 archivos `.Designer.cs`, generados por Visual Studio; solo se reconocen
Paquetes que importan	`System.Data.SqlClient` (hablar con SQL Server), `BCrypt.Net-Next` (contraseñas), `iTextSharp` (PDF) y `DataVisualization` (gráficos del inicio)

2.1 Los archivos que debes reconocer

Archivo	Qué hace
`Program.cs`	Punto de entrada: decide si toca el asistente de primer uso, el login o el panel

Archivo	Qué hace
`Conexion.cs`	Arma la conexión a SQL Server y le pone la etiqueta del usuario
`Datos.cs`	Los cinco métodos con los que todo el programa consulta y escribe
`Sesion.cs`	Guarda en memoria quién inició sesión y con qué rol
`PasswordHasher.cs` y `ServicioUsuarios.cs`	Hash de contraseñas con BCrypt; claves temporales y reglas de contraseña
`frmLogin`, `frmRecuperarContraseña`, `frmConfiguracionInicial`	Las tres puertas de entrada
`frmDashboard`	El panel principal con el menú lateral y la navegación
`frmGestionPersonal`, `frmAgregarPersonal` y las demás `frmGestion...`	Las pantallas de cada módulo
`ControlesTema.cs`, `ItemMenuLateral.cs`	Los controles que se dibujan por código

2.2 Las cuatro capas

Arriba, las **pantallas** (`frmXxx`): reciben clics y muestran datos. Debajo, los **servicios y el acceso a datos** (`Datos`, `Conexion`, `ServicioUsuarios`): saben hablar con la base y con el exterior. A un lado, la **capa de dibujo** (controles personalizados): sin lógica de negocio, pero es lo que da identidad visual. Al fondo, **SQL Server**, donde viven las reglas y los datos. Una pantalla casi nunca habla con SQL Server directamente: pasa por `Datos`.

2.3 Cómo leer cualquier pantalla

Todas siguen el mismo ritmo, así que aprender una es aprender todas. Hazte tres preguntas, en este orden:

1. **¿Qué consulta carga los datos?** Normalmente en el evento `Load` o en un método `Cargar...`.
2. **¿Qué evento dispara cada acción?** Un clic, un cambio en el buscador, una selección en la tabla.
3. **¿Qué se valida antes de guardar y quién guarda?** La pantalla misma o un procedimiento almacenado.

Con esas tres respuestas puedes explicar cualquier formulario sin haberlo memorizado.

3. Lo imprescindible de C# y WinForms

3.1 Clases, objetos y `static`

Una clase es un **molde** y un objeto es una pieza hecha con ese molde. `ItemMenuLateral` es un molde: cada opción del menú (Inicio, Activos, Personal…) es un objeto con su propio texto, icono y estado. Una clase `static` (`Datos`, `Conexion`, `Sesion`, `PasswordHasher`) **no se instancia**: es un conjunto de herramientas o valores compartidos, y se usa directamente: `Datos.Tabla(...)`.

`Sesion` es estática porque solo hay un usuario conectado a la vez y cualquier pantalla necesita saber quién es (`Sesion.Usuario`, `Sesion.Rol`). El precio es que lo estático es estado global; para un sistema de escritorio de un solo usuario es una decisión razonable.

3.2 Propiedades: estado, `Invalidate()` y repintado

Una propiedad es un campo con control de acceso. En `ItemMenuLateral`:

```
private bool activo = false;
public bool Activo
{
    get { return activo; }
    set { if (activo != value) { activo = value; Invalidate(); } }
}
```

Nadie toca el campo; todos pasan por la propiedad, y su `set` hace algo más: llama a `Invalidate()`, que le pide a Windows que vuelva a dibujar el control. **El patrón que se repite en todo el diseño es: estado → `Invalidate()` → `OnPaint` → pantalla.**
Modificadores: `public` lo ve cualquiera, `private` solo la clase, `internal` solo el proyecto; `const` es un valor fijo (`MaxFallos = 5`).

3.3 Herencia, `override` y `partial`

`BotonTema : Button` es un botón normal al que se le añade la paleta; `GridOscuro : DataGridView`, una tabla con colores propios. Con `override` se cambia lo heredado: sobrescribir `OnPaint` permite dibujar a tu manera, y `base.OnPaint(e)` significa «haz primero lo que haría el padre».

`partial` significa que una clase está repartida en dos archivos: `frmLogin.cs`, que escribes tú, y `frmLogin.Designer.cs`, que escribe Visual Studio al arrastrar controles. Su `InitializeComponent()` crea los botones y cuadros de texto. Los dos archivos son **una sola clase**.

3.4 Ciclo de vida de un formulario y `ShowDialog`

El constructor llama a `InitializeComponent()`; después se dispara `Load`, el lugar natural para cargar datos; al cerrarse, `FormClosing`. `Show()` abre una ventana y sigue; **`ShowDialog()` la abre como modal**: la ejecución se detiene ahí hasta que se cierre y devuelve un `DialogResult`.

```
if (frm.ShowDialog(this) == DialogResult.OK) MostrarEmpleados();
```

Si el formulario de alta se cerró con `OK` (guardó bien), la lista se recarga; si se canceló, no pasa nada.

3.5 Eventos, `+=` y delegados

Algo ocurre (un clic) y el programa reacciona. Un **manejador** es un método con firma `(object sender, EventArgs e)` que se conecta con `+=`: `timerMenu.Tick += timerMenu_Tick;`. Una lambda (`(s, ev) => ...`) es lo mismo escrito en línea, sin nombre. Un **delegado** es una variable que guarda una función: `Func<Form>` es «una función que devuelve un formulario». Así, `Navegar(itmPersonal, () => new frmGestionPersonal())` le pasa a `Navegar` **la receta** para crear la pantalla, no la pantalla ya hecha, y `Navegar` la ejecuta dentro de un `try/catch`.

3.6 `using`: liberar recursos

Conexiones, pinceles y archivos retienen recursos que no son solo memoria. `using (...) { ... }` los libera al salir del bloque, **aunque ocurra un error**: es un `try/finally` que no puedes olvidar. Una conexión que no se cierra se acumula y, tras unas cuantas, la base deja de aceptar más. Por eso toda la clase `Datos` envuelve la conexión en `using`.

3.7 Excepciones

`try` envuelve lo que puede fallar y `catch` decide qué hacer. **El orden de los `catch` importa**: de lo más específico a lo más general (`catch (SqlException)` antes de `catch (Exception)`), y el compilador no deja ponerlos al revés. `finally` se ejecuta siempre (el login lo usa para volver a habilitar el botón). `throw;` relanza la misma excepción tras deshacer algo. Un `catch { }` vacío es una decisión consciente solo para cosas secundarias, como registrar un evento de auditoría.

3.8 `null`, `DBNull` y conversiones

SQL tiene `NULL`, C# tiene `null`, y entre ambos está `DBNull.Value`, la forma en que ADO.NET representa un `NULL` de la base. **Al escribir**, `Datos.P` convierte con `valor ?? DBNull.Value`. **Al leer**, hay que preguntar antes si la columna puede ser nula:

`reader["Telefono"] != DBNull.Value ? ... : ""`. Los valores llegan como `object`, por eso se convierten con `Convert.ToInt32(...)`, `Convert.ToDecimal(...)`.

3.9 Hilos y `async` en una frase

Windows Forms dibuja en un único hilo; si una tarea lenta lo bloquea (enviar un correo), la ventana se congela. Por eso el envío de correo usa `Task.Run` y la pantalla lo espera con `await`: es el único tipo de tarea asíncrona del sistema, porque es lo único que depende de internet.

4. SQL dentro de C#: las dos consultas del examen

Esta es la sección más importante: aquí el C# y la base de datos se tocan. Para C#, una consulta SQL es solo un **texto** que se manda por una conexión; la base lo ejecuta y devuelve un resultado. Casi cualquier pregunta sobre «cómo funciona esto» termina en una consulta dentro de un método.

4.1 El texto: la cadena literal con arroba

Las consultas se escriben con `@"..."` (cadena literal): respeta los saltos de línea, así que la consulta se lee en varias líneas. **Hay dos arrobas distintas**: la de afuera (`@"..."`) es de C#; la de adentro (`@idEmpleadoExcluido`) es de SQL y marca un **parámetro**, un hueco que C# rellena después con un valor.

4.2 Primera consulta: ¿ya existe un director?

Vive en `frmAgregarPersonal` y responde sí o no: «¿hay ya otro director registrado?».

```
string query = @"SELECT COUNT(*)
    FROM Empleados e
    INNER JOIN Cargos c ON e.IdCargo = c.IdCargo
    WHERE LOWER(c.NombreCargo) LIKE '%director%'
      AND LOWER(c.NombreCargo) NOT LIKE '%sub%'
      AND e.IdEmpleado <> @idEmpleadoExcluido";
cmd.Parameters.AddWithValue("@idEmpleadoExcluido", idEmpleadoExcluido);
int count = Convert.ToInt32(cmd.ExecuteScalar());
return count > 0;
```

Se lee de abajo hacia arriba: `FROM Empleados e` elige la tabla; el `INNER JOIN` trae el **nombre** del cargo (el empleado solo guarda su número); el `WHERE` descarta filas y deja las que contienen «director», no contienen «sub» (para no contar la Subdirectora) y no son el empleado que se está editando; `COUNT(*)` no devuelve filas, devuelve **cuántas** quedaron.

Al registrar uno nuevo se manda `0` como Id (nadie lo tiene, no se excluye a nadie); al editar al propio director se manda su Id, para que no se cuente a sí mismo. `ExecuteScalar()` devuelve la primera columna de la primera fila.

La pantalla repite la misma regla en C# antes de llamar: `nombreCargo.ToLower().Contains("director") && !nombreCargo.ToLower().Contains("sub")`, igual que el `LIKE` y el `NOT LIKE`. **Limitaciones honestas:** no mira el estado del empleado (un director dado de baja seguiría contando); depende del **texto** del nombre del cargo («Director de Sistemas» contaría como el director); y la regla la impone la pantalla, no la base.

4.3 Segunda consulta: la lista de personal

```
dtEmpleados = Datos.Tabla(@"SELECT e.IdEmpleado, e.Nombre, c.NombreCargo AS
Cargo,
    est.NombreEstado AS Estado, e.FechaContratacion, p.SalarioNeto AS
[Salario Neto], ...
  FROM Empleados e
  INNER JOIN Cargos c            ON e.IdCargo   = c.IdCargo
  INNER JOIN EstadosEmpleado est ON e.IdEstado  = est.IdEstado
  LEFT  JOIN Planillas p         ON e.IdPlanilla = p.IdPlanilla
  WHERE est.NombreEstado != 'Inactivo'
  ORDER BY e.Nombre");
```

Esta no responde sí o no: **trae una tabla**. Cada `AS` pone el nombre con el que aparecerá la columna (los corchetes de `[Salario Neto]` son porque lleva un espacio). Tres uniones traen cargo, estado y estructura salarial. `WHERE ... != 'Inactivo'` deja fuera a los dados de baja. `Datos.Tabla` ejecuta la consulta con un `SqlDataAdapter` y **llena un `DataTable` en memoria**. No lleva parámetros porque `'Inactivo'` es una constante del programa, no algo que escribe el usuario.

4.4 `INNER JOIN` y `LEFT JOIN`

`INNER JOIN` es estricto: solo trae las filas que **encuentran pareja**. `LEFT JOIN` trae todas las de la izquierda y rellena con `NULL` si no hay pareja. **Regla:** `LEFT JOIN` cuando el dato de la derecha es opcional; `INNER JOIN` cuando es obligatorio. El cargo y el estado deben existir (el formulario no deja guardar sin ellos); la planilla puede faltar, y con `INNER JOIN` ese empleado desaparecería de la lista. (Matiz honesto: en la tabla esas columnas aceptan `NULL`; quien garantiza que siempre tengan valor es la pantalla.)

4.5 `LIKE`, comodines y parámetros

`LIKE` compara contra un **patrón**: `%` es «cualquier cantidad de caracteres». `'doc%'` empieza con doc; `'%dor'` termina así; `'%director%'` lo contiene. `LOWER()` ignora las mayúsculas. Con texto del usuario hay dos formas seguras: los comodines **en SQL** alrededor del parámetro (`LIKE '%' + LOWER(TRIM(@criterio)) + '%'`) o **dentro del valor** del parámetro (`AddWithValue("@Criterio", "%" + texto + "%")`). En las dos, lo que escribió el usuario viaja **como valor**.

4.6 Parámetros e inyección SQL

Un parámetro viaja **por separado** del texto de la consulta: la base compila la consulta con huecos y luego inserta los valores como datos, nunca como código. La alternativa mala sería pegar texto: `"... WHERE Usuario = '" + txtUsuario.Text + "'"`. Si alguien escribe `' OR '1'='1`, la condición se vuelve verdadera para todas las filas: eso es una **inyección SQL**. Con parámetros, ese mismo texto se busca literalmente como un usuario rarísimo y no se encuentra a nadie.

Tres formas de crearlos, todas seguras: `AddWithValue("@x", v)`; `Datos.P("@x", v)`, que además convierte `null` en `DBNull`; y `Datos.PTexto("@x", texto)`, que guarda un cuadro vacío como `NULL`. **Concatenar está bien si lo que se pega es una constante del programa** (el filtro elegido por un `switch` en Préstamos, o `, DUI = @dui` si es administrador); lo que escribe el usuario siempre va como parámetro.

4.7 Las cuatro formas de ejecutar

Método	Devuelve	Se usa para
`ExecuteScalar()`	Un solo valor (primera columna, primera fila)	`COUNT`, `SUM`, un `SELECT TOP 1`, un Id recién creado
`ExecuteNonQuery()`	El **número de filas afectadas**	`INSERT`, `UPDATE`, `DELETE`, procedimientos
`ExecuteReader()`	Un cursor que lees fila por fila	Leer una vez (lo usa el login)
`SqlDataAdapter.Fill`	Todo el resultado en un `DataTable`	Listas para mostrar (lo que hay detrás de `Datos.Tabla`)

`Datos.Tabla` no llama a `Open()` porque `Fill` abre y cierra la conexión; con las otras tres sí hay que abrirla.

4.8 Resumir en la propia consulta

`COUNT(*)` cuenta filas; `SUM(CASE WHEN Alerta = 'Atrasado' THEN 1 ELSE 0 END)` cuenta solo las que cumplen una condición (`CASE` es el `if` de SQL), así dos contadores salen con **un solo viaje** a la base. `ISNULL(valor, 0)` cambia un `NULL` por un reemplazo, necesario porque `SUM` sobre cero filas da `NULL` y en C# eso rompería `Convert.ToInt32`. `GROUP BY` agrupa para resumir por grupo.

4.9 Escribir datos

`INSERT`, `UPDATE` y `DELETE` se ejecutan con `ExecuteNonQuery()`, cuyo número devuelto (filas afectadas) confirma que algo cambió: `return cmd.ExecuteNonQuery() > 0;`. Para saber el Id que la base le puso a un registro nuevo, el `INSERT` termina con un segundo `SELECT`: `SELECT CAST(SCOPE_IDENTITY() AS INT);` y se ejecuta con `ExecuteScalar()` (se usa `SCOPE_IDENTITY()` y no `@@IDENTITY` porque este último puede devolver el Id de otra tabla si un trigger inserta algo). Para llamar a un procedimiento: `Datos.Procedimiento("sp_RegistrarPrestamo", Datos.P(...), ...)` si no esperas respuesta, o `Datos.Tabla("EXEC dbo.sp_GenerarPlanilla @Periodo = @p, ...")` si termina con un `SELECT` que necesitas leer.

4.10 `DataTable`, `DataView` y el filtro en memoria

Un `DataTable` es una tabla **en la memoria del programa**, ya desconectada de la base. Un `DataView` es una ventana sobre ella con su propio filtro. El buscador no consulta otra vez a SQL Server: arma un texto y lo asigna a `vista.RowFilter`, y la cuadrícula se actualiza al instante. **`RowFilter` no es SQL**, es un lenguaje de expresiones de .NET (`Activo = true`), por eso existe la función `Escapar` que protege las comillas y los comodines. No es inyección: la expresión nunca llega a la base.

5. La conexión, la clase `Datos`, errores y transacciones

5.1 `Conexion`: dónde está la base y quién es el usuario

`Conexion.Conectar()` tiene una sola responsabilidad: **fabricar conexiones** (sin abrir). La cadena apunta a `(localdb)\MSSQLLocalDB`, una versión liviana de SQL Server que se levanta bajo demanda, sin instalar un servidor, y a la base `Gestion_administrativa_y_escolar`. Usa `Integrated Security=true` (autenticación de Windows), así que **no hay ninguna contraseña de base de datos escrita en el código**. El límite: funciona donde el servidor es local o Windows reconoce al usuario.

5.2 Cómo sabe la base quién hizo un cambio

La auditoría la hacen los **triggers**, pero un trigger no sabe quién está sentado frente al programa: para la base, todas las conexiones son del mismo usuario de Windows. La solución: `Conectar()` suscribe a cada conexión el evento `StateChange`, y cuando pasa de **cerrada a abierta** ejecuta `sp_set_session_context @key = N'Usuario', @value = @u` con `Sesion.Usuario`. Los triggers leen ese valor con `SESSION_CONTEXT(N'Usuario')` y lo escriben en la auditoría. Ninguna pantalla tiene que hacer nada: **es automático y transversal**. Si no hay sesión (modo Debug), la auditoría registra «(sistema)».

5.3 La clase `Datos`

`Datos` es una clase estática con las herramientas con que todo el programa consulta y escribe:

Método	Para qué
`Datos.Tabla(sql, parámetros)`	Devuelve un `DataTable` (listas)
`Datos.Escalar(sql, ...)`	Devuelve un solo valor
`Datos.Ejecutar(sql, ...)`	`INSERT`, `UPDATE`, `DELETE`
`Datos.Procedimiento(nombre, ...)`	Llama a un procedimiento almacenado
`Datos.P(...)` y `Datos.PTexto(...)`	Crean parámetros (`null` → `DBNull`; texto vacío → `NULL`)

Cada método **abre su propia conexión y la cierra** dentro de un `using`. No es lento gracias al **pool de conexiones**: al cerrar, la conexión física no se destruye, se guarda en una reserva y se reutiliza. El costo es que **dos llamadas seguidas a `Datos` no pueden compartir una transacción**; si necesitas atomicidad hay que abrir una sola conexión con `SqlTransaction`, o meter todo en un procedimiento almacenado.

5.4 Transacción desde C#: guardar un empleado

Guardar un empleado implica la fila, su foto y sus materias; si fallara la foto quedaría un registro a medias. Por eso `frmAgregarPersonal` hace:

```
using (SqlConnection con = Conexion.Conectar())
{
    con.Open();
    using (SqlTransaction tx = con.BeginTransaction())
    {
        try
        {
            int idEmp = GuardarEmpleado(con, tx);
            GuardarFoto(con, tx, idEmp);
            GuardarMaterias(con, tx, idEmp);
            tx.Commit();
        }
        catch { tx.Rollback(); throw; }
    }
}
```

Una transacción es un grupo de operaciones que se confirman **todas** (`Commit`) o se deshacen **todas** (`Rollback`). Cada método auxiliar recibe la misma conexión y la misma transacción, y crea sus comandos con `new SqlCommand(texto, con, tx)` : ese tercer argumento los une a la transacción. Si algo falla, `Rollback()` deshace todo (incluido el empleado) y `throw;` relanza el error para que la pantalla lo muestre.

5.5 Errores en los procedimientos

Los procedimientos usan `SET XACT_ABORT ON` (cualquier error cancela la transacción entera), `BEGIN TRY ... BEGIN CATCH` y, en el `CATCH` , `IF @@TRANCOUNT > 0 ROLLBACK; THROW;` . Los errores de negocio se lanzan a propósito: `THROW 50034, 'No se puede prestar una unidad en mal estado...', 1;` . Desde C# llegan como una **`SqlException`** : su `Message` trae el texto (ya escrito en lenguaje claro) y su `Number` el número.

`Number`	Significa
50000 en adelante	Errores propios de las reglas del sistema (`THROW`). Se muestran tal cual
2601 y 2627	Se intentó guardar un valor que ya existe (índice o restricción `UNIQUE`)
547	Violación de llave foránea o de `CHECK` (p. ej. borrar algo que está en uso)

5.6 Cuatro formas de mostrar un error

1. **Mostrar el mensaje tal cual:** `MessageBox.Show(... + ex.Message)` (Préstamos). Funciona porque el procedimiento escribió el mensaje para personas.
2. **Filtrar con `when`:** `catch (SqlException ex) when (ex.Number == 2601 || ex.Number == 2627)`.
3. **Un `switch` sobre `ex.Number`** (`frmRegistrarPago`): marca con `ErrorProvider` el campo exacto y le pone el foco.
4. **Buscar el nombre de la restricción en el mensaje** (`ex.Message.Contains("UQ_Empleados_Nombre")`): más práctico, pero frágil si se renombra el índice.

Las restricciones llevan nombres claros (`PK_`, `UQ_`, `FK_`, `CK_`) justamente porque SQL Server los incluye en el mensaje de error y el programa los lee.

5.7 Si SQL Server está apagado

Al arrancar, `Program.Main` verifica la configuración dentro de un `try/catch`: si la base no responde, muestra «No se pudo verificar la configuración» y termina. En el login, una bandera `conexionFallida` evita **contar el intento como contraseña equivocada**. En las demás pantallas, cada carga tiene su `try/catch` que muestra el error en un aviso sin cerrar el programa.

6. La lógica de las pantallas

Una pantalla es una **clase parcial que reacciona a eventos**. Siguen todas la misma plantilla, y por eso aprender una es aprender todas.

6.1 La plantilla de carga y la bandera `cargando`

En el evento `Load` la pantalla configura sus controles y llama a métodos `Cargar...()` que ejecutan una consulta, llenan un `DataTable` y lo enlazan a la tabla. El problema: cuando el código llena un combo o asigna `SelectedIndex`, el control **dispara su evento `SelectedIndexChanged` como si lo hubiera hecho una persona**, y si ese evento lanza una consulta se ejecutaría a medias. La solución es una bandera: `bool previo = cargando;` `cargando = true;` antes de llenar, `cargando = previo;` al terminar, y `if (cargando) return;` al inicio de los manejadores sensibles. Se restaura el valor **anterior** (no se pone `false` a ciegas) para que funcione si un método que ya estaba cargando llama a otro. Es la misma idea que `enviando` y `guardando` en otras pantallas.

6.2 Combos que muestran un texto y guardan un identificador

La persona ve «Aula 3 · Proyector · Bueno», pero el código necesita el número de la unidad. Se logra con `DisplayMember` / `ValueMember` o con objetos `ElementoLista` (`Id` , `Texto`) que sobrescriben `ToString()` . Un ayudante, `IdSeleccionado(combo)` , devuelve un `int?` (nulo si no hay selección). Así el texto es para la persona y el número para la base.

6.3 Validar en la pantalla y otra vez en la base

La validación de la pantalla es **comodidad**: avisa al instante y marca el campo exacto. La de la base es **garantía**: se aplica venga el dato de donde venga. La regla de oro: *nunca confiar solo en la pantalla*. El patrón del formulario: `errorProvider.Clear(); bool hayError = false;` ; para cada campo inválido, `errorProvider.SetError(control, "mensaje"); hayError = true;` ; al final `if (hayError) return;` . El `ErrorProvider` dibuja un icono rojo junto al control. La clave es que **no se detiene en el primer error**: marca todos a la vez para que el usuario corrija de una vez. Después vienen las reglas de negocio que requieren consulta (¿ya hay un director?) y, al final, el guardado: lo barato y local primero, lo que escribe en la base al final.

6.4 Crear y editar en el mismo formulario

`frmAgregarPersonal` recibe `idEmpleadoEditar` . Si vale `0` , es un empleado nuevo; si trae un número, es edición y carga los datos de esa persona. Al guardar, `if (idEmpleadoEditar == 0)` decide entre `INSERT` y `UPDATE` . Las validaciones, los campos y el aspecto existen **una sola vez**. La pantalla de la lista lo abre con `ShowDialog` y, si devuelve `OK` , recarga con `MostrarEmpleados()` : la lista nunca se actualiza «por su cuenta», siempre se recarga desde la base tras un cambio.

6.5 El CRUD de Personal

- **Leer:** `frmGestionPersonal` carga la lista (la consulta de la sección 4.3) y el buscador filtra en memoria.
- **Crear y actualizar:** el formulario `frmAgregarPersonal` , con validación y transacción (sección 5.4).
- **Eliminar:** no borra, es una **baja lógica** (`UPDATE ... SET IdEstado = 2`). El empleado desaparece de la lista pero conserva pagos, ausencias y auditoría. Antes, la pantalla avisa si tiene pagos pendientes, préstamos activos o salones a su cargo, y el botón «No» está seleccionado por defecto.

6.6 Buscador y filtros combinados

Cada filtro activo aporta **una condición** a una `List<string> cond`, y se unen con `string.Join(" AND ", cond)` para asignarlas a `vista.RowFilter`. No hay un `if` por cada combinación; agregar un filtro nuevo es agregar una línea. Si ningún filtro está activo, `cond` queda vacío y se muestran todas las filas. Los eventos `TextChanged` de los controles llaman al mismo método, por eso el resultado cambia mientras escribes. El texto del usuario pasa por `Escapar` antes de entrar a la expresión.

6.7 Controlar lo que se escribe

Hay dos niveles. **Impedir la tecla**: en `KeyPress`, `e.Handled = true` impide escribir un carácter inválido (el teléfono solo acepta dígitos, `+`, `-` y espacio; el nombre solo letras). **Validar al guardar** (`hayError`), necesario porque se puede pegar texto saltando el `KeyPress`. El DUI valida su formato `00000000-0` y su **dígito verificador** (se multiplican los ocho primeros dígitos por 9, 8, 7… 2, se suman y se calcula `(10 - suma % 10) % 10`). Solo se pide a quien edita como administrador.

6.8 El panel principal: pantalla dentro de pantalla

`frmDashboard` tiene el menú lateral y un contenedor (`pnlContenedor`). Al hacer clic en un ítem **no se abre una ventana**: `AbrirFormularioEnContenedor` elimina y libera (`Dispose`) lo que hubiera, configura el formulario nuevo con `TopLevel = false`, `FormBorderStyle = None` y `Dock = Fill`, y lo agrega. Ventajas: se siente una sola aplicación y no se acumulan pantallas en memoria; la contrapartida es que cada vez que vuelves a una sección **se recrea y vuelve a consultar la base** (datos frescos, estado perdido). `Navegar(item, Func<Form>)` lo orquesta dentro de un `try/catch`. Las **insignias** del menú (reportes abiertos, pagos pendientes) las recalcula un `Timer` cada 60 segundos, con un `catch` que las deja en cero: una insignia es decoración, no debe interrumpir con errores. Atajos: **F1** abre el manual y **Ctrl + B** pliega el menú, con `ProcessCmdKey`.

7. Seguridad en el código

La seguridad es un indicador de la rúbrica, así que esta sección merece tu mejor práctica en voz alta.

7.1 La sesión y el control de acceso

`Sesion` es una clase estática que guarda `Usuario`, `Rol` y `NombreCompleto` mientras el programa está abierto. El control tiene tres capas: el **login** (solo entran administradores con la cuenta activa), la **sesión** y, en el panel, `AplicarPermisosSidebar`, que oculta

secciones si el rol no es Administrador (hoy solo entran administradores; es una preparación para otros roles, y ocultar es una medida de interfaz: la protección real de datos tendría que estar también en la base). **Debug y Release:** en `Program.Main` una directiva `#if DEBUG` hace que, compilando para desarrollo, el programa abra el panel **sin login** y asigne el rol Administrador; en Release se abre el login. Por eso **se presenta siempre en Release**.

7.2 El login, paso a paso

En `btnIngresar_Click` de `frmLogin`:

1. **Cláusula de guarda:** si falta usuario o contraseña, marca el campo y sale.
2. **Bloqueo:** si ese usuario está bloqueado (hora futura en un diccionario), rechaza **sin consultar la base**.
3. Llama a `ValidarCredenciales`, que hace dos cosas **siempre, en este orden**: **autenticar** (¿la clave es correcta? por defecto *no*; se verifica con BCrypt y una clave vacía nunca coincide) y **autorizar** (¿la cuenta está activa y el rol es Administrador?). Cada rechazo deja su motivo (`inactivo`, `no_admin`) para el mensaje y la bitácora.
4. Si todo está bien: llena `Sesion`, registra el acceso con `sp_RegistrarAcceso` (dentro de un `try/catch` vacío a propósito: si la bitácora falla no debe impedir entrar), comprueba si la clave es temporal y abre el panel con `ShowDialog`.
5. Si falla: registra el motivo, limpia la contraseña y marca los campos. Un servidor caído (`conexionFallida`) **no cuenta** como intento fallido.

La contraseña nunca se compara en SQL: la base entrega el hash y es C# quien verifica. Existe una rama de compatibilidad para claves antiguas en texto plano: si el valor guardado no empieza con `$2` y coincide, se acepta **solo si la cuenta pasa también la autorización** y, entonces, se reemplaza por su hash.

7.3 El bloqueo por intentos fallidos

Dos diccionarios `static` en memoria: `fallosPorUsuario` y `bloqueadoHasta`, con el usuario en minúsculas como llave. Cada contraseña equivocada suma un fallo; desde el tercero el mensaje dice cuántos quedan; al quinto (`MaxFallos = 5`) se bloquea 60 segundos. Solo cuentan las contraseñas equivocadas, no las cuentas desactivadas. **Límites:** vive en memoria (reiniciar el programa lo borra) y es por usuario, no por equipo.

7.4 BCrypt: por qué es un hash y no cifrado

Un **hash** es de un solo sentido: no se puede volver a la contraseña. **Cifrar** es reversible, y quien roba la clave obtiene todas las contraseñas. BCrypt es un hash diseñado para ser **lento a propósito** y trae un **salt** (valor aleatorio distinto por contraseña, así dos usuarios

con la misma clave tienen hashes distintos) y un **factor de costo**. El resultado es una cadena de unos 60 caracteres que empieza con `$2`. En el código:

```
PasswordHasher.Hash(clave)      // BCrypt.Net.BCrypt.HashPassword
PasswordHasher.Verificar(clave, hash) // lee el salt y el costo del propio hash
```

Se usa `Hash` donde nace una contraseña (asistente, cambio de clave, clave temporal, recuperación). **La contraseña nunca viaja a SQL Server ni queda en una consulta.** Para demostrarlo: `SELECT TOP 3 Usuario, Clave FROM dbo.Usuarios;` muestra cadenas ilegibles que empiezan con `$2`.

7.5 Contraseñas temporales y códigos

`GenerarClaveTemporal` crea 10 caracteres con al menos una mayúscula, una minúscula y un número, sin los que se confunden (`I` , `O` , `l` , `o` , `0` , `1`). El azar sale de `RandomNumberGenerator` , el generador **criptográfico**, no de `Random` (que es predecible). La clave temporal se guarda como hash, se marca `DebeCambiarClave = 1` y el login obliga a cambiarla antes de entrar.

7.6 Recuperación por correo

El usuario escribe su correo; si existe, se genera un **código de seis dígitos** criptográfico, se envía por correo de forma asíncrona y se guarda en memoria con su vencimiento (15 minutos). Al restablecer se revisa, en orden: que las contraseñas coincidan, que el código no haya expirado y que sea el correcto; a los 5 intentos incorrectos se invalida. **Límites honestos:** la pantalla dice «Este correo no está registrado» (revela qué correos existen); el código se verifica en el propio programa; y las reglas de contraseña no son iguales en todas las pantallas (6 caracteres en recuperación, 8 en el asistente, 8 con letras y números en la gestión de usuarios).

7.7 El asistente de primer uso

`Program.Main` llama a `EsPrimerUso()` (`SELECT COUNT(*) FROM ConfiguracionEmpresa`): si está vacía, abre el asistente y, si lo cancelan, el programa termina: **no se puede usar sin configurar**. Son tres pasos: datos de la institución, primer administrador y verificación del correo con código (se puede omitir). `GuardarInicial` **calcula el hash en C#** y llama a `sp_GuardarConfiguracionInicial` , que guarda todo en una transacción. Las banderas `enviando` y `guardando` evitan dobles clics.

7.8 Lo que se debe reconocer

El servicio de correo usa un **token** escrito como constante en el código fuente (se envía por HTTP a un script de Google): es un secreto compartido que no debe viajar con el código; la mejora es moverlo a un archivo de configuración y regenerarlo si el repositorio es público.

8. Diseño por código

El diseño es lo primero que ve el jurado. Nada de esto viene «de fábrica» con Windows Forms (que por defecto es gris): es código escrito a propósito y es donde más se nota el trabajo del equipo. Habla de **ideas**, no de líneas sueltas.

8.1 Cómo se dibuja en Windows Forms

Cuando Windows necesita mostrar un control llama a su método `OnPaint` y le pasa `e.Graphics`, el **lienzo** (línea `DrawLine`, formas `DrawEllipse`, rellenos `FillPath`, curvas `DrawBezier`; la librería se llama GDI+). Para dibujar necesitas un **lápiz** (`Pen`, contorno) y un **pincel** (`SolidBrush`, relleno), que son recursos y se crean dentro de `using`. El texto se pinta con `TextRenderer.DrawText`.

La regla de oro: el dibujo se hace *dentro* de `OnPaint`, nunca «por fuera». Si algo debe cambiar de aspecto, **cambias el estado** (`hover = true`) y llamas a `Invalidate()`, que le pide a Windows volver a llamar a `OnPaint`. El aspecto siempre es una función del estado actual: **estado → `Invalidate()` → `OnPaint` → pantalla**.

8.2 Los iconos del menú: líneas sobre una cuadrícula de 24 por 24

Viven en `IconosMenu.Dibujar(g, nombre, caja, color)`. Todos se diseñan sobre una **cuadrícula imaginaria de 24 × 24 unidades** y el método la amplía o reduce:

```
g.TranslateTransform(caja.X, caja.Y);   // mueve el origen al cuadrado
float k = caja.Width / 24f;
g.ScaleTransform(k, k);                 // 24 unidades = ancho real
```

Al empezar hace `g.Save()` y al terminar restaura, para que las transformaciones no «salgan» y descoloquen otros dibujos. El lápiz es de 1,8 con puntas y esquinas **redondeadas** (`LineCap.Round`), lo que da a todos los iconos un trazo coherente. Un `switch` por nombre usa primitivas simples: la casa de «inicio» son tres `DrawLines` (tejado, paredes y puerta); «personal» son dos círculos (`DrawEllipse`) y dos medias circunferencias (`DrawArc(..., 180, 180)`).

¿Por qué por código? Escala sin perder nitidez (es geometría, no píxeles); **el color es un parámetro** (el mismo dibujo es gris, blanco, amarillo o rojo según el estado, sin cuatro imágenes); no hay archivos externos que se pierdan; y todos comparten el mismo estilo.

8.3 `ItemMenuLateral`: un control que cambia según su estado

Hereda de `Control` (no de `Button`), así que lo pinta todo él. En el constructor activa `UserPaint` («yo me pinto»), `OptimizedDoubleBuffer`, `AllPaintingInWmPaint` y `ResizeRedraw`. Guarda su estado (`activo`, `hover`, `peligro`, `compacto`, `insignia`), y cada propiedad llama a `Invalidate()` en su `set`. `OnPaint` sigue siempre el mismo orden: limpia con el fondo del padre; si está activo o con el ratón encima, dibuja un rectángulo redondeado detrás; si está activo, la **barra amarilla** a la izquierda; elige colores con cadenas de condiciones (gris deshabilitado, amarillo activo, blanco con ratón); dibuja el icono, el texto y la **insignia**. La insignia es una cápsula: se mide el texto, se calcula el ancho y se rellena una forma redondeada con radio igual a la mitad de la altura; el número se limita a «99+». En modo **compacto** (menú plegado) solo queda un punto de color.

8.4 Esquinas redondeadas

GDI+ no trae un rectángulo redondeado, así que `Redondeado(r, radio)` arma un `GraphicsPath` con **cuatro arcos**, uno por esquina:

```
p.AddArc(r.X,         r.Y,          d, d, 180, 90);  // superior izquierda
p.AddArc(r.Right - d, r.Y,          d, d, 270, 90);  // superior derecha
p.AddArc(r.Right - d, r.Bottom - d, d, d,   0, 90);  // inferior derecha
p.AddArc(r.X,         r.Bottom - d, d, d,  90, 90);  // inferior izquierda
p.CloseFigure();
```

`d` es el diámetro (radio × 2); cada `AddArc` es un cuarto de circunferencia y GDI+ une los arcos con líneas rectas. Se rellena con `FillPath` o se contornea con `DrawPath`. **`Region` es distinto:** *recorta* el control mismo (así se redondea la tarjeta del login, `pnlTarjeta.Region = new Region(...)`), pero sin suavizado de bordes; `FillPath` solo *pinta* una forma dentro del control.

8.5 `PaletaSistema` y los controles del tema

`PaletaSistema` es una clase estática con nueve colores con **nombre por su uso** (`Fondo` casi negro, `Tarjeta`, `Campo`, `Borde`, `Amarillo` de acento, `Gris`, `Rojo`, `Texto`, `Tenue`). Cambiar la identidad visual sería tocar un archivo. *Honestidad:* varias clases (`ItemMenuLateral`, `GridOscuro`) declaran sus propios colores cercanos en lugar de leer la paleta; es una deuda de orden, no un fallo.

El sistema crea **clases propias que heredan de los controles estándar** y ya traen el estilo puesto: `BotonTema` (propiedad `Tipo`: Principal amarillo, Secundario gris, Peligro rojo; el efecto al pasar el ratón usa `FlatAppearance`), `PanelTarjeta` (fondo y borde fino), `GridOscuro` (tabla con filas alternas, resaltado de la fila bajo el ratón con `InvalidateRow`, franja amarilla en la fila seleccionada y barras de desplazamiento propias, `BarraFina`, superpuestas a las nativas) y `ComboOscuro` (la lista con `OwnerDrawFixed` y el cuadro cerrado interceptando `WM_PAINT` en `WndProc`). Para temas que Windows no deja cambiar con propiedades (el calendario desplegable) se usa **P/Invoke**: `SetWindowTheme` y `SendMessage` de `user32.dll` y `uxtheme.dll`. **Qué se gana:** el estilo se escribe una vez y se usa en todo el sistema.

8.6 `CellFormatting` frente a `CellPainting`

`CellFormatting` se ejecuta antes de mostrar la celda y deja **cambiar sus propiedades** (color de fondo o de letra): así la tabla de préstamos pinta en rojo oscuro los «Atrasado» y en ámbar los «Por vencer». `CellPainting` permite **dibujar la celda tú mismo**, y es lo que hace falta para una cápsula. En `PintarPildora`: `e.Paint(...)` pide que se pinte todo menos el texto, `e.Handled = true` avisa «ésta la dibujo yo», se mide el texto, se rellena una forma redondeada con el color base a **transparencia parcial** (`Color.FromArgb(55, color)`) y se escribe el texto encima. Si solo cambias un color usa `CellFormatting`; si necesitas una forma nueva, `CellPainting`.

8.7 Doble búfer y el login

Sin ayuda, repintar muestra pasos intermedios y se percibe **parpadeo**. El doble búfer arma el dibujo en una imagen oculta y la copia de una vez (`OptimizedDoubleBuffer`); `ResizeRedraw` repinta al cambiar de tamaño. En `frmLogin` hay detalles que se ven: cada campo es un panel que dibuja una **línea inferior** cuyo color decide una cadena de condiciones (rojo si hay error, amarillo si tiene el foco); el **ojo** que muestra la contraseña son dos curvas de Bézier (`DrawBezier`) y una pupila, y alterna `PasswordChar`; y se sobrescribe `CreateParams` con `WS_EX_COMPOSITED` para reducir el parpadeo.

8.8 Límites del diseño (dilos tú)

La paleta no está totalmente centralizada; el login y el panel tienen `AutoScaleMode = None`, así que el diseño no se reescala solo con el DPI del monitor; y en algunas tablas la última fila puede verse cortada cuando la altura no es múltiplo del alto de fila. Ninguno compromete el funcionamiento. La página responsiva de la rúbrica es el trabajo del equipo web; aquí se priorizó un diseño cuidado y consistente para un tamaño de ventana controlado.

9. Tres flujos de punta a punta

La mejor prueba de que entiendes el sistema es **seguir un dato desde el clic hasta la base y de vuelta a la pantalla**. Estos tres recorridos tocan casi todo lo importante.

9.1 Flujo 1: de escribir la contraseña a ver el panel

1. `btnIngresar_Click` recoge y limpia (`Trim()`); si falta algo, marca el campo y sale (cláusula de guarda).
2. Revisa el **bloqueo** del usuario (en minúsculas) **sin tocar la base**.
3. Deshabilita el botón y lo repinta (`Refresh()`); un `try/finally` lo restaura pase lo que pase.
4. `ValidarCredenciales` : consulta con parámetros, lee la fila, **autentica** con BCrypt y **autoriza** (activo y Administrador). Devuelve el rol o `null` , y deja el motivo en `motivoRechazo` .
5. Si hay rol: borra los contadores de fallos, llena `Sesion` , registra el acceso con `sp_RegistrarAcceso` (con un `catch` vacío a propósito), revisa si la clave es temporal y abre el panel con `ShowDialog` dentro de un `using` .
6. Al volver, si `Sesion.Rol` quedó vacío (cerró sesión), muestra de nuevo el login; si no, cierra el programa.

Si falla: registra el motivo exacto, suma un fallo **solo si fue contraseña equivocada** y al quinto bloquea 60 segundos. Si falló la conexión, no cuenta ni registra nada.

9.2 Flujo 2: registrar un préstamo de equipo

Cruza cuatro capas: la pantalla (`frmGestionActivos` , pestaña Préstamos), `Datos` , el procedimiento `sp_RegistrarPrestamo` con sus tablas y triggers, y la vista `vw_Prestamos` .

- **Antes del clic:** el combo de unidades **solo ofrece lo prestable**: su consulta filtra por equipo, excluye los estados no prestables con `NOT IN (...)` y las unidades con préstamo activo con `NOT EXISTS` . Los combos usan `ElementoLista` (texto para la persona, Id para el código).
- **El clic (`btnPrestar_Click`):** valida en pantalla (equipo, unidad, responsable, fecha límite futura) con `ErrorProvider` y `if (error) return;` ; luego llama a `Datos.Procedimiento("sp_RegistrarPrestamo", ...)` . Esta validación es **de cortesía**, no la defensa real.
- **En SQL Server:** el procedimiento valida el empleado (existe, activo, cargo coherente con el tipo) y **toma el nombre del responsable del registro del empleado, no de la pantalla**; valida la unidad (existe, no está en mal estado, no está prestada) y recién entonces hace el `INSERT` . Repite reglas que la pantalla ya aplicó: es **defensa en**

profundidad, porque otra persona pudo prestar esa unidad entre la carga y el clic, y como respaldo existe un índice único filtrado que impide dos préstamos activos de la misma unidad.

- **El trigger:** `trg_Aud_Prestamos` escribe en `Auditoria` el JSON del registro y el usuario leído de `SESSION_CONTEXT`.
- **Vuelta a C#:** si el procedimiento lanza un error llega como `SqlException` y se muestra `ex.Message`; si sale bien, limpia y recarga con `CargarPrestamos()`. Esa consulta lee la vista `vw_Prestamos`, que calcula la columna `Alerta` con `GETDATE()` (**Atrasado** si el límite pasó, **Por vencer** si vence en menos de dos días) y `dgvPrestamos_CellFormatting` la pinta. **La regla de negocio está en SQL; el color está en C#.**

9.3 Flujo 3: generar la planilla del mes

- **Selector de mes:** `frmGenerarPlanilla` (un formulario hecho solo por código) arma un `DataTable` con el mes actual y los 11 anteriores; no hay meses futuros.
- **Vista previa:** ejecuta `EXEC dbo.sp_GenerarPlanilla @Periodo = @p, @Usuario = @u, @Simular = 1`. El procedimiento calcula todo lo que haría y **no inserta nada**: devuelve cuántos pagos crearía, cuántos ya existen, cuántos empleados no tienen salario y si el mes está cerrado. Los errores de mes futuro o cerrado solo se lanzan al ejecutar de verdad, por eso la vista previa **puede mostrar** «mes cerrado» en lugar de fallar.
- **Generación real:** con `@Simular = 0`, en una **transacción** y con un solo `INSERT ... SELECT`, crea un pago «Pendiente» por cada empleado activo sin pago y con salario (con el descuento por ausencias topado para no superar base más bono). Ejecutarlo dos veces **no duplica nada**: es **idempotente**.
- **¿Por qué simular en SQL y no calcular la vista previa en C#?** Porque habría dos copias de la misma lógica que tarde o temprano se contradirían. Con `@Simular` hay una sola fuente de verdad.

9.4 Receta para contar cualquier flujo

Cinco frases: *qué botón se pulsa y en qué método; qué valida la pantalla; qué consulta o procedimiento se ejecuta y con qué parámetros; qué reglas y qué tablas toca la base; y cómo se refresca la pantalla o cómo se muestra el error.* Si puedes responderlas para estos tres flujos, puedes hacerlo para casi cualquier pantalla.

10. Preguntas y respuestas esenciales

Son las 40 preguntas que más pesan de las 73 de la guía completa (secciones 12 a 17), con cada respuesta reducida a lo que necesitas decir en 30 a 60 segundos. Las marcadas con ★ son las que casi seguro caen: empieza por ellas. Cómo estudiarlas: lee la pregunta, tapa la respuesta, dila con tus palabras y compara. Si te trabas, ese punto es el que debes releer; al final de cada respuesta va el número de la pregunta en la guía completa por si quieres el detalle.

10.1 C# y Windows Forms

P1. ¿Qué diferencia hay entre una clase `static` (`Datos`) y una normal (`ItemMenuLateral`)? ¿Por qué `Sesion` es estática?

Una clase normal es un molde: se crea con `new` y cada objeto guarda su propio estado (cada ítem del menú recuerda su texto, su icono y si está seleccionado). Una clase `static` no se instancia: sus miembros pertenecen a la clase (`Datos.Tabla(...)`) y se usa para herramientas que no recuerdan nada, como `Datos` o `PasswordHasher`. `Sesion` es estática porque guarda algo único y visible desde cualquier pantalla: el usuario conectado. Costo honesto: es estado global y dificulta las pruebas, pero para una aplicación de escritorio de un solo usuario es razonable. *(Completa: 12.1)*

P2. ¿Qué es una propiedad, por qué el `set` de `Activo` llama a `Invalidate()` y qué es `override OnPaint`?

Una propiedad es un campo con `get` y `set` donde puedes ejecutar código al leer o escribir. En `ItemMenuLateral`, el `set` de `Activo` cambia el valor y llama a `Invalidate()`, que le pide a Windows «vuélveme a pintar»; Windows entonces ejecuta `OnPaint`, que la clase sobrescribe (`override`) para dibujar con `e.Graphics` según el estado actual. El flujo que debes recordar, porque se repite en todos los controles propios: **estado → `Invalidate()` → `OnPaint` → pantalla**. `base.OnPaint(e)` se llama si quieres dibujar encima del dibujo estándar y no se llama si lo reemplazas por completo. *(Completa: 12.2, 12.4)*

P3. ¿Qué significa `partial` y para qué existe `Designer.cs`?

`partial` es una clase repartida en varios archivos que el compilador junta como uno solo. `frmLogin.Designer.cs` lo escribe Visual Studio (su `InitializeComponent()` crea y coloca cada control); `frmLogin.cs` es tuyo y lleva la lógica. Así el diseñador no pisa tu código ni tú el suyo. En `frmGestionActivos` se usa además para dividir un formulario enorme por temas (activos, préstamos, mantenimiento). *(Completa: 12.3)*

P4. ¿Qué diferencia hay entre `Show()` y `ShowDialog()`? ¿Qué es `DialogResult`?

`Show()` abre la ventana y el código sigue de largo. `ShowDialog()` la abre como **modal**: la línea que lo llama espera, la ventana de atrás queda bloqueada y, al cerrarse, devuelve un `DialogResult` (`OK`, `Cancel`…). Ejemplo: `if (frm.ShowDialog(this) == DialogResult.OK) MostrarEmpleados();` significa «si el usuario guardó, recarga la lista». El login abre el panel con `ShowDialog()`, así que la línea siguiente solo corre cuando el panel se cierra. *(Completa: 12.5)*

P5. ¿Qué es un evento? ¿Qué significa `+=`? ¿Qué es `Func<Form>`?

Un evento es un aviso que emite un objeto (`Click`, `Tick`); el manejador es el método que reacciona, y `+=` lo suscribe (aquí no suma: agrega a la lista de interesados). `=>` define una función anónima (lambda). `Func<Form>` es un **delegado**: una función guardada en una variable que no recibe nada y devuelve un `Form`. `Navegar(item, () => new frmGestionPersonal())` recibe la **receta** para fabricar la pantalla y no la pantalla ya hecha; así decide cuándo construirla, dentro de un `try/catch`. *(Completa: 12.6)*

P6. ¿Para qué sirve `using`?

Garantiza que se llame a `Dispose()` al salir del bloque, pase lo que pase (es un `try/finally` abreviado). Se usa con recursos que el recolector de basura no libera a tiempo: conexiones, lápices y pinceles de dibujo, formularios. Por eso `Datos` envuelve cada conexión en `using`. Sin él la fuga es silenciosa: tras muchas consultas se agotaría el *pool* de conexiones y el sistema fallaría al azar. *(Completa: 12.7)*

P7. ¿Cómo funciona `try / catch / finally`? ¿Importa el orden de los `catch`? ¿Qué hace `throw;`?

`try` envuelve lo que puede fallar; el control salta al primer `catch` que coincida; `finally` se ejecuta siempre (por ejemplo, rehabilitar el botón). El orden **sí importa**: primero el específico (`SqlException`) y después el general (`Exception`); al revés ni compila. `throw;` relanza la misma excepción conservando su origen, a diferencia de `throw ex;`, que lo borra. Un `catch` vacío es sospechoso salvo que se justifique: en `RegistrarAcceso` es porque la bitácora no debe impedir entrar, y en las insignias del menú porque son decoración. *(Completa: 12.8, 15.2)*

P8. ¿Qué es `DBNull` y por qué `Datos.P` usa `??`?

El `NULL` de SQL no es el `null` de C#. Al **escribir**, un parámetro con `null` falla; hay que enviar `DBNull.Value`, y `Datos.P` lo resuelve una vez: `valor ?? DBNull.Value` («si es nulo, usa esto»). Al **leer**, una columna `NULL` llega como `DBNull.Value`: `ToString()` funciona, pero `Convert.ToInt32(...)` lanzaría una excepción, por eso se pregunta antes (`reader["x"] == DBNull.Value`). *(Completa: 12.9)*

P9. ¿Para qué sirve la bandera `cargando`?

Para decir «estoy llenando la pantalla, no reacciones». Cuando el código llena un combo o asigna `SelectedIndex`, el control dispara `SelectedIndexChanged` como si lo hubiera hecho una persona; con `if (cargando) return;` el manejador lo ignora. Se guarda y restaura el valor previo (`bool previo = cargando; ... cargando = previo;`) para que funcione si un método que ya cargaba llama a otro que también lo hace. En una frase: en Windows Forms el código y el usuario disparan los mismos eventos. *(Completa: 12.10)*

10.2 SQL dentro de C#

P10 ★. Explica la consulta de `YaExisteDirector`.

```
SELECT COUNT(*)
FROM Empleados e
INNER JOIN Cargos c ON e.IdCargo = c.IdCargo
WHERE LOWER(c.NombreCargo) LIKE '%director%'
  AND LOWER(c.NombreCargo) NOT LIKE '%sub%'
  AND e.IdEmpleado <> @idEmpleadoExcluido
```

Pregunta si **ya hay otro director**. El `JOIN` une cada empleado con su cargo para tener el nombre; el `WHERE` se queda con los cargos que contienen «director» pero no «sub» (así la «Subdirectora» no cuenta) y descarta al empleado que se está editando (`@idEmpleadoExcluido` vale 0 si es nuevo); `COUNT(*)` devuelve cuántos quedaron. En C# se ejecuta con `ExecuteScalar()` y el método devuelve `count > 0`. Límites que conviene decir tú: no mira el estado (un director dado de baja seguiría contando), se basa en el **texto** del cargo y la base no impone la regla, solo la pantalla. *(Completa: 13.1)*

P11 ★. ¿Por qué `INNER JOIN` con cargos y estados pero `LEFT JOIN` con planillas?

`INNER JOIN` solo devuelve una fila si encuentra pareja; `LEFT JOIN` devuelve **todas** las de la izquierda y rellena con `NULL` si no hay pareja. Cargo y estado son obligatorios para el formulario, así que `INNER`; la planilla es opcional (el sistema contempla empleados «sin salario asignado»), así que `LEFT`: el empleado sin planilla no desaparece de la lista y se puede encontrar para asignársela. Regla: **`LEFT` cuando el dato de la derecha es opcional, `INNER` cuando es obligatorio**. Precisión honesta: en la tabla `IdCargo` e `IdEstado` aceptan `NULL`; quien garantiza que siempre tengan valor es el formulario. *(Completa: 13.2, 13.3)*

P12 ★. ¿Qué es un parámetro? ¿Qué es una inyección SQL? Hay consultas con `+`: ¿no es inyección?

Un parámetro (`@x`) es un hueco: C# manda por separado el texto de la consulta y los valores, y la base los trata como **datos, nunca como código**. Si se pegara el texto (`"... WHERE Usuario = '" + txt.Text + "'"`), escribir `' OR '1'='1` volvería la condición verdadera para todas las filas: eso es inyección SQL. En el proyecto hay tres formas de

crear parámetros, equivalentes en seguridad: `AddWithValue`, `Datos.P` (convierte `null` en `DBNull`) y `Datos.PTexto` (un cuadro vacío se guarda como `NULL`). Un `+` dentro de una consulta solo es seguro si lo pegado es una **constante del programa** (el fragmento elegido por un `switch` en los préstamos, o `, DUI = @dui` si es administrador); lo que escribe el usuario siempre viaja como parámetro. La pregunta de control es: «¿de dónde sale lo que se concatena?». Con `LIKE`, el comodín se pone en la consulta (`'%' + @c + '%'`) o dentro del valor. *(Completa: 13.4, 13.5, 13.13)*

P13. ¿Qué diferencia hay entre `ExecuteScalar`, `ExecuteNonQuery`, `ExecuteReader` y `Fill`? ¿Cómo se obtiene el Id recién insertado?

`ExecuteScalar` devuelve **un valor** (primera columna de la primera fila: `COUNT`, `SCOPE_IDENTITY`; `null` si no hay filas). `ExecuteNonQuery` ejecuta `INSERT/UPDATE/DELETE` y devuelve el **número de filas afectadas** (0 significa que no cambió nada, por ejemplo porque otro usuario ya lo borró). `ExecuteReader` lee **fila por fila** (el login). `SqlDataAdapter.Fill` trae **todo** a un `DataTable` (es lo que hay detrás de `Datos.Tabla`). Para el Id nuevo: `INSERT ...; SELECT CAST(SCOPE_IDENTITY() AS INT);` con `ExecuteScalar`. Se usa `SCOPE_IDENTITY()` y no `@@IDENTITY` porque este puede devolver el Id de otra tabla cuando un trigger de auditoría insertó algo. *(Completa: 13.6, 13.9, 13.10)*

P14. ¿Qué es un `DataTable` y qué hace `RowFilter`? ¿Cómo se combinan buscador y filtros?

Un `DataTable` es una tabla en la memoria del programa, ya desconectada de la base. La lista se trae **una vez** y el buscador la filtra con `RowFilter` de un `DataView`, sin volver a la base. `RowFilter` **no es SQL** (es un lenguaje de expresiones de .NET), por eso el texto del usuario pasa por `Escapar` (comillas, `[`, `%`, `*`); no es inyección porque la expresión nunca llega a la base. Los filtros se combinan con un patrón: cada control activo aporta **una condición** a una `List<string>` y al final `RowFilter = string.Join(" AND ", cond)`; sin filtros queda vacío y se ven todas las filas. Límite: con cientos de miles de filas habría que filtrar en SQL con paginación. *(Completa: 13.7, 15.4)*

P15. ¿Cómo se llama a un procedimiento almacenado desde C#?

De dos maneras. `Datos.Procedimiento("sp_RegistrarPrestamo", Datos.P(...), ...)` cuando **no esperas respuesta**. `Datos.Tabla("EXEC dbo.sp_GenerarPlanilla @Periodo = @p, @Simular = 1", ...)` cuando el procedimiento **termina con un `SELECT`** que quieres leer. En ambos casos los `THROW` del procedimiento llegan a C# como `SqlException`. Criterio de diseño: lecturas y filtros en C# (cambian seguido); operaciones que cambian datos y tienen reglas en procedimientos (transacción y regla juntas, iguales desde cualquier pantalla). *(Completa: 13.12, 17.7)*

10.3 Conexión, errores y transacciones

P16 ★. ¿Qué hace `Conexion`? ¿Cómo sabe la base quién hizo un cambio (`StateChange`)?

`Conexion.Conectar()` fabrica la conexión: LocalDB (`(localdb)\MSSQLLocalDB`, una versión liviana de SQL Server sin instalar servidor), la base `Gestion_administrativa_y_escolar` e `Integrated Security`, que es **autenticación de Windows**: no hay ninguna contraseña de base de datos en el código. Además suscribe a cada conexión el evento `StateChange`: cuando pasa de cerrada a abierta, ejecuta `sp_set_session_context` con `Sesion.Usuario`; los triggers de auditoría leen `SESSION_CONTEXT(N'Usuario')` y lo escriben en `Auditoria`. Es automático para todas las pantallas; sin sesión el cambio queda como «(sistema)». *(Completa: 14.1, 14.2)*

P17 ★. ¿Por qué cada llamada abre y cierra su conexión? ¿Qué es una transacción y cómo se usa?

Abrir tarde y cerrar pronto es la práctica recomendada, y no es lento por el **pool de conexiones**: al cerrar, la conexión física se guarda y se reutiliza. El costo es que dos llamadas de `Datos` no comparten transacción. Una transacción agrupa operaciones para que salgan **todas o ninguna**. Guardar un empleado (fila, foto y materias) usa una sola conexión y se une cada comando a la transacción con `new SqlCommand(texto, con, tx)`:

```
using (SqlConnection con = Conexion.Conectar()) {
  con.Open();
  using (SqlTransaction tx = con.BeginTransaction()) {
    try { int id = GuardarEmpleado(con, tx);
          GuardarFoto(con, tx, id); GuardarMaterias(con, tx, id);
          tx.Commit(); }
    catch { tx.Rollback(); throw; }
  }
}
```

Si algo falla, `Rollback()` deshace todo y `throw;` deja que la pantalla muestre el error. La otra solución es meter todo en un procedimiento almacenado, que ya es atómico. *(Completa: 14.3, 14.4)*

P18 ★. ¿Para qué sirven `XACT_ABORT`, `TRY/CATCH` y `THROW`? ¿Qué es una `SqlException` y qué significan 50000+, 2601/2627 y 547?

En los procedimientos: `SET XACT_ABORT ON` cancela la transacción ante cualquier error; en el `CATCH` se hace `IF @@TRANCOUNT > 0 ROLLBACK; THROW;`, que deshace y relanza el error original. Las reglas de negocio se lanzan con `THROW 50034, 'mensaje', 1;`. En C# llegan como `SqlException` con `Message` (el texto) y `Number`: **50000 o más** son reglas nuestras (se muestran tal cual), **2601 y 2627** son un valor único repetido (DUI, nombre),

547 es una llave foránea o un `CHECK` (borrar algo que está en uso). Idea para cerrar: la base nunca deja pasar datos incoherentes y C# traduce el motivo a lenguaje humano. *(Completa: 14.5, 14.6)*

P19. ¿Cómo se muestran los errores? ¿Cómo funciona `ErrorProvider` con `hayError`? ¿Por qué se valida en C# y en la base?

Hay cuatro estilos, del más simple al más fino: mostrar `ex.Message`; `catch (SqlException ex) when (ex.Number == 2601 || ex.Number == 2627)`; un `switch` sobre `ex.Number` que además marca el campo exacto con `ErrorProvider` (`frmRegistrarPago`); y buscar el nombre de la restricción en el mensaje (`UX_Empleados_DUI`; es el más frágil si se renombra el índice). El patrón `hayError` hace `errorProvider.Clear()`, revisa **todos** los campos marcando cada uno inválido sin detenerse en el primero y al final muestra un solo aviso. Se valida en C# por **comodidad** (avisa al instante) y en la base por **garantía** (vale venga el dato de donde venga): nunca confiar solo en la pantalla. *(Completa: 14.7, 14.8)*

P20. ¿Cómo se elimina algo que está en uso?

Tres estrategias. **Intentar y traducir el error**: el `DELETE` de una estructura de salario en uso falla con el 547 y la pantalla dice «está en uso». **Instrucción selectiva**: `Salon.QuitarActivo` borra solo las unidades sin historial con tres `NOT EXISTS` y avisa de cuántas no pudo retirar. **Baja lógica**: `UPDATE Empleados SET IdEstado = 2`; el registro conserva sus pagos, préstamos y auditoría y solo deja de salir en las listas. Eliminar a un empleado desde la pantalla es una baja lógica, con confirmación que trae «No» seleccionado por defecto. *(Completa: 14.9, 15.3)*

P21. ¿Qué pasa si dos personas editan el mismo registro a la vez?

Es una aplicación de escritorio con base local, poco concurrente, pero lo crítico está protegido en la base: `sp_RegistrarDevolucion` bloquea la fila con `UPDLOCK`, el índice único filtrado `UX_Prestamos_UnidadActiva` impide dos préstamos activos de la misma unidad y los `UNIQUE` frenan duplicados. Lo que **no** hay es concurrencia optimista en las ediciones: gana quien guarda último, sin aviso. La mejora concreta es una columna `rowversion` y un `UPDATE ... WHERE Id = @id AND Version = @v`; si devuelve 0 filas, avisar al usuario. *(Completa: 14.11)*

10.4 Pantallas y seguridad

P22. ¿Cómo funciona el panel principal? ¿Por qué las pantallas se incrustan?

`frmDashboard` tiene el menú a la izquierda y un contenedor a la derecha. `Navegar` libera lo que hubiera en el contenedor, crea el formulario con la receta y lo incrusta con `TopLevel = false`, `FormBorderStyle = None` y `Dock = Fill`; marca el ítem activo y actualiza las insignias, todo dentro de un `try/catch`. Ventaja: parece una sola aplicación

y no se acumulan ventanas. Costo: al volver a una sección se recrea y se pierde su estado, pero los datos siempre están frescos. Un `Timer` de 60 segundos actualiza las insignias (se detiene al cerrar); **F1** abre el manual y **Ctrl+B** pliega el menú. *(Completa: 15.1, 15.2)*

P23. Explica un CRUD completo con Personal.

Leer: `frmGestionPersonal` carga la lista en un `DataTable` y la filtra en memoria. **Crear y actualizar** comparten `frmAgregarPersonal` en «modo dual»: si `idEmpleadoEditar == 0` hace `INSERT`, si trae un número hace `UPDATE`, con validaciones, transacción y `SCOPE_IDENTITY()`. Si `ShowDialog` devuelve `OK`, la lista se recarga desde la base. **Eliminar**: baja lógica. Las validaciones y el aspecto existen una sola vez porque el formulario es el mismo. *(Completa: 15.3)*

P24. ¿Cómo se restringe lo que se escribe? ¿Qué pasa con el DUI?

Dos niveles: en `KeyPress`, `e.Handled = true` impide la tecla inválida (el teléfono solo acepta dígitos, `+`, `-` y espacio), y al guardar se vuelve a validar porque se puede pegar texto. Para el DUI, el formulario pone el guion solo, comprueba el formato `00000000-0` y el **dígito verificador** (los ocho primeros dígitos por 9, 8, 7…2, se suman y se calcula `(10 - suma % 10) % 10`). En la versión de `entrega_final` ese cálculo ya está activo: se quitó un `return true;` que lo dejaba inalcanzable. Detrás está la base: `CK_Empleados_DUI` (formato) y `UX_Empleados_DUI` (único). *(Completa: 15.5)*

P25 ★. ¿Cómo se controla quién entra? ¿Por qué Debug se comporta distinto?

Tres capas: el **login** (solo entran usuarios activos con rol Administrador), la clase **`Sesion`** (guarda usuario, rol y nombre) y `AplicarPermisosSidebar` (oculta secciones a otros roles: es una medida de interfaz, no de seguridad real). El flujo del login es: comprobar bloqueo → leer la fila con parámetro → `BCrypt.Verify` en C# → comprobar activo → comprobar rol → registrar el acceso, siempre con el mensaje genérico «usuario o contraseña incorrectos» para no dar pistas. En la versión corregida se **autentica y se autoriza siempre**: la rama para claves antiguas en texto plano solo acepta valores que no empiezan por `$2`, nunca una clave vacía, exige activo y rol, y recién después migra la clave a hash. Sobre Debug: `#if DEBUG` en `Program.Main` salta el login para comodidad del equipo; por eso hay que **presentar y entregar siempre la compilación Release**. *(Completa: 15.6, 17.2)*

P26 ★. ¿Qué es BCrypt? ¿Por qué hash y no cifrado?

Un **hash** es de un solo sentido: no se puede volver del hash a la contraseña, ni siquiera quien administra la base. **Cifrar** es reversible con una clave, y si se roba la clave caen todas. Para validar no hace falta recuperar la contraseña: se calcula el hash de lo que escribió el usuario y se compara. BCrypt es **lento a propósito**, mete un **salt** aleatorio por contraseña (dos usuarios con la misma clave tienen hashes distintos y las tablas precalculadas no sirven) y tiene un **factor de costo** ajustable. El resultado es una cadena

de unos 60 caracteres que empieza por `$2` (`$2a$11$...`) y contiene versión, costo, salt y hash. `PasswordHasher.Hash` y `Verificar` envuelven la librería, y la comparación ocurre en C#, nunca en SQL. *(Completa: 15.7)*

P27. ¿Cómo se genera la contraseña temporal? ¿Por qué no `Random`?

`GenerarClaveTemporal` crea 10 caracteres con al menos una mayúscula, una minúscula y un número, sin los que se confunden al leer (`I`, `O`, `l`, `o`, `0`, `1`). Usa `RandomNumberGenerator`, el generador **criptográfico**, porque `Random` es predecible a partir de su semilla. Un muestreo por rechazo evita el sesgo del módulo, y Fisher–Yates mezcla la clave para que no empiece siempre con la misma estructura. Se guarda como hash con `DebeCambiarClave = 1`, y el login obliga a cambiarla antes de entrar. *(Completa: 15.8)*

P28. Explica el bloqueo por intentos y la recuperación de contraseña.

Bloqueo: dos diccionarios estáticos en memoria, por usuario en minúsculas; con 5 fallos (`MaxFallos`) se bloquea 60 segundos (`SegundosBloqueo`) y el intento se rechaza antes de consultar la base; un acierto limpia el contador. Solo cuentan las contraseñas equivocadas (no la cuenta inactiva ni una base caída, que activa `conexionFallida`). Límites: está en memoria (reabrir el programa lo borra), es por usuario y no por equipo. **Recuperación**: correo → código de 6 dígitos criptográfico → caduca a los 15 minutos, 5 intentos; al restablecer se comprueba que las claves coincidan, que no haya expirado y que el código sea el correcto, y se guarda el hash. Límite: decir «correo no registrado» revela qué correos existen. *(Completa: 15.9, 15.10)*

P29. ¿Cómo funciona el asistente de primer uso? ¿Y el envío de correos? ¿Qué riesgo tiene?

`Program.Main` llama a `EsPrimerUso()`: si `ConfiguracionEmpresa` está vacía abre el asistente de 3 pasos (institución, primer administrador y verificación del correo, que se puede omitir); si lo cancelan el programa termina, así que **no se puede usar sin configurar**. El hash se calcula en C# y `sp_GuardarConfiguracionInicial` guarda todo en una transacción; las banderas `enviando` y `guardando` evitan dobles clics. Los correos salen con un **HTTP POST** a un servicio de Google Apps Script (el programa no guarda la contraseña de ningún correo), con `async/Task.Run` para no congelar la ventana. Riesgo: la dirección y el *token* de ese servicio están como constantes en el código fuente; la mitigación es regenerar el token y sacarlo a un archivo de configuración. *(Completa: 15.11, 15.12, 12.11)*

10.5 Diseño por código

P30. ¿Cómo se dibuja en Windows Forms? ¿Cómo se hacen los iconos, las esquinas redondeadas y la paleta?

Todo control se pinta en `OnPaint` con `e.Graphics` (GDI+): un `Pen` para contornos y un `SolidBrush` para rellenos, ambos dentro de `using`. Los **iconos** del menú están dibujados por código sobre una cuadrícula de 24×24, con `TranslateTransform` y `ScaleTransform` para ajustarlos a cualquier tamaño: se ven nítidos, cambian de color con el estado y no dependen de archivos. Las **esquinas redondeadas** se construyen con un `GraphicsPath` y cuatro `AddArc` (ángulos 180, 270, 0 y 90) que luego se rellena con `FillPath`; `Region` en cambio recorta el control entero, sin suavizado. Los colores están en **`PaletaSistema`** (fondo oscuro, texto claro y un solo acento amarillo), y `BotonTema`, `PanelTarjeta`, `GridOscuro` y `ComboOscuro` heredan de controles estándar para reutilizar el estilo en todas las pantallas. *(Completa: 16.1 a 16.6)*

P31. ¿Qué diferencia hay entre `CellFormatting` y `CellPainting`? ¿Cómo es `GridOscuro`?

`CellFormatting` cambia las **propiedades** de la celda antes de mostrarla (color de fondo, color de letra): así se pinta en rojo un préstamo atrasado. `CellPainting` te deja **dibujar la celda tú mismo** (con `e.Handled = true`): así se hacen las cápsulas de estado (`PintarPildora`: relleno translúcido con alfa 55 y el texto encima). `GridOscuro` hereda de `DataGridView`, configura todo en el constructor, resalta la fila bajo el ratón invalidando solo las dos filas afectadas y cubre las barras de desplazamiento nativas con `BarraFina`. Para los desplegables y el calendario se intercepta `WM_PAINT` en `WndProc` y se usa P/Invoke (`SetWindowTheme`), que llama funciones nativas de Windows. *(Completa: 16.8 a 16.10)*

P32. ¿Qué es el doble búfer? ¿Es «responsive» el programa de escritorio?

El doble búfer arma el dibujo completo en memoria y lo muestra de una vez, para que no haya parpadeo (`OptimizedDoubleBuffer`, `AllPaintingInWmPaint`, `ResizeRedraw`; en el login también `WS_EX_COMPOSITED`). Sobre el diseño adaptable: una aplicación WinForms no reorganiza su contenido como una página web. Las pantallas incrustadas usan `Dock = Fill` y algunas `Anchor`, y el login y el panel arrancan **maximizados**, con botones de minimizar, maximizar y cerrar. Tienen `AutoScaleMode = None` para controlar el dibujo al píxel, a cambio de que con escalados de pantalla muy distintos el tamaño pueda no ser ideal. La parte responsiva que pide la rúbrica es la página web del equipo. *(Completa: 16.7, 16.13)*

10.6 Preguntas trampa, arquitectura y mejoras

P33 ★. Demuéstrame que no hay inyección SQL.

Con una demostración: en el login escribe `' OR '1'='1` como usuario y cualquier contraseña; responde «Usuario o contraseña incorrectos». La consulta es `WHERE u.Usuario = @user`, así que ese texto se busca literalmente como un nombre de usuario

raro y no existe. Luego explica tu criterio de revisión: todo valor que escribe una persona entra como parámetro; donde hay un `+`, se pega una constante del programa; `RowFilter` no es SQL y nunca llega a la base. Límite honesto: la protección depende de la **disciplina** de usar parámetros en cada consulta nueva, no de una barrera técnica. *(Completa: 17.1)*

P34 ★. ¿Se puede entrar sin conocer la contraseña?

Por el camino normal, no: BCrypt más bloqueo por intentos. La vía real es el **modo Debug** (`#if DEBUG` abre el panel sin login): se evita presentando siempre en Release y comprobando que pide usuario. La otra brecha, la rama de claves antiguas en texto plano que se saltaba los controles de cuenta activa y de rol, ya está corregida en `entrega_final`: ahora se autentica y se autoriza siempre, se rechaza la clave vacía y la migración a hash ocurre solo tras autorizar. Para comprobar tus datos: `SELECT Usuario FROM Usuarios WHERE Clave NOT LIKE '$2%'`; si no devuelve filas, todas las claves ya están en hash. *(Completa: 17.2)*

P35. ¿Hay inconsistencias, código duplicado o código sin uso?

Sí, y se nombran con calma. Las reglas de contraseña difieren por pantalla (asistente: 8; recuperación: 6; `ValidarClaveNueva`: 8 con letras y números; el asistente tampoco hace `Trim()`). `CorreoValido` está repetido en cuatro formularios y `GenerarCodigo` en dos. Hay clases de una etapa anterior que las pantallas actuales no llaman (`Empleado.cs`, `AsignarCargo.cs`, `Planilla.cs`, `Dbsetup.cs`). Respuesta: es código heredado identificado, no se borró antes de la entrega para no arriesgar nada a última hora, y la solución es **una regla, un lugar**: una función de validación compartida y utilidades comunes en `Modelos`. *(Completa: 17.3, 17.4)*

P36. ¿Usa el patrón MVC? ¿Qué patrones reconoces?

No en sentido estricto. Hay dos proyectos (`Vistass` con las pantallas y `Modelos` con conexión, datos y servicios), pero los formularios mezclan vista, controlador y consulta (*code-behind*, el estilo por defecto de Windows Forms). Lo mejor separado son las **reglas críticas, que viven en la base**. Patrones reales, con su archivo: **Observador** (los eventos), **Fachada** (`Datos`, `PasswordHasher`), **estado global** tipo Singleton (`Sesion`), **herencia especializada** (`BotonTema`, `GridOscuro`) y **delegados** (`Navegar` con `Func<Form>`). No inventes: si no puedes señalar dónde está, no lo menciones. *(Completa: 17.5, 17.6)*

P37. ¿Por qué ADO.NET y no Entity Framework? ¿Por qué Windows Forms y no WPF o web?

ADO.NET con SQL a mano da control y transparencia (ves exactamente qué se ejecuta) y es lo más didáctico; Entity Framework esconde el SQL y agrega migraciones y contexto, algo arriesgado a última hora. El SQL va en C# para lecturas y filtros y en procedimientos para operaciones con reglas y transacción. Windows Forms es la opción de escritorio más directa, con base local (LocalDB) y sin servidor, y GDI+ permite una identidad visual

propia. WPF es más rico pero más difícil, y una aplicación web exige servidor y seguridad de red; la web responsiva de la rúbrica es el trabajo del equipo web. Límite de WinForms: está en mantenimiento y maneja peor las densidades de pantalla. *(Completa: 17.7, 17.8)*

P38. ¿Hay pruebas automatizadas? ¿Qué mejorarías con una semana más?

No hay un proyecto de pruebas automatizadas: las pruebas fueron manuales por pantalla y de los procedimientos con `EXEC` (datos válidos e inválidos). Mejora: pruebas unitarias de la lógica pura (DUI, clave temporal, `Escapar`) y de integración de los procedimientos (por ejemplo, que un préstamo de una unidad ya prestada lance el 50035). Lista priorizada: **1)** presentar en Release y revisar que no queden claves sin hash; **2)** sacar el token del correo del código y regenerarlo; **3)** unificar la regla de contraseñas; **4)** reglas de cargo por identificador y no por texto; **5)** `rowversion` para concurrencia; **6)** bloqueo de login persistente en la base; **7)** una sola paleta; **8)** limpieza del código heredado y proyecto de pruebas. *(Completa: 17.9, 17.10)*

P39. Si piden agregar «Nivel educativo» al empleado, ¿qué tocas y en qué orden?

De abajo hacia arriba: **1)** la base, con `ALTER TABLE Empleados ADD ...` (tipo, si admite `NULL`, `CHECK`); **2)** la vista `vw_EmpleadosDetalle` y las consultas que deban mostrarlo; **3)** la auditoría, regenerando el trigger `trg_Aud_Empleados` porque lista las columnas; **4)** el C#: el control en el formulario, la validación en el bloque `hayError`, el campo en el `INSERT` y el `UPDATE` con su parámetro, la carga en modo edición y la consulta de la lista; **5)** los reportes, si debe salir; **6)** probar crear, editar, buscar y revisar la auditoría. Demuestra que un dato recorre base, vista, auditoría, formulario, lista y reportes. *(Completa: 17.11)*

P40 ★. Explícame el sistema en dos minutos. ¿Y si me preguntan algo que no sé?

Esqueleto de cinco partes: qué es, cómo está hecho, cómo se protege, qué lo distingue y qué le falta. «Es un sistema de escritorio en C# y Windows Forms para administrar una institución escolar: personal, planillas, activos y préstamos, salones y reportes de daño. Por debajo hay una base en SQL Server con tablas relacionadas, vistas, procedimientos y triggers de auditoría. Son dos proyectos, uno con las pantallas y otro con el acceso a datos y los servicios; todo pasa por la clase `Datos`, que abre una conexión corta y usa siempre parámetros. La seguridad tiene capas: BCrypt, bloqueo por intentos, recuperación con código que caduca y un asistente de primer uso; las reglas importantes viven en la base. El diseño es un tema oscuro con acento amarillo hecho por código con GDI+. Conocemos sus límites y cómo corregirlos.» Si no sabes algo: **reconócelo con precisión**, razona con lo que sí sabes, abre el código y léelo con el profesor; **nunca inventes**. *(Completa: 17.12, 17.13)*

11. Glosario mínimo y repaso final de 20 minutos

Esta sección es para el final: el glosario te sirve para comprobar que cada palabra técnica que aparece en el código te suena a algo concreto, la lista de defectos es lo que debes decir tú antes de que te lo pregunten, y el repaso cronometrado cierra el estudio.

11.1 Glosario mínimo: las 35 palabras que debes poder explicar en una frase

Término	En una frase
Clase `static`	Clase que no se instancia; sus miembros se usan directo (`Datos.Tabla`, `Sesion.Usuario`).
Propiedad	Campo con `get` y `set`; el `set` puede ejecutar código, como llamar a `Invalidate()`.
Evento y manejador	Aviso que emite un objeto (`Click`, `Tick`) y el método que reacciona; se conectan con `+=`.
Delegado (`Func<Form>`)	Una función guardada en una variable; `Navegar` recibe la receta para crear la pantalla.
`partial`	Una clase repartida en varios archivos (`frmLogin.cs` y `frmLogin.Designer.cs`).
`override` y `base`	Reemplazar o extender un método heredado; `base.OnPaint(e)` ejecuta primero el del padre.
`using` y `Dispose`	Libera el recurso (conexión, lápiz) al salir del bloque, aunque haya error.
`DBNull`	El `NULL` de SQL visto desde C#; `Datos.P` lo envía con `??` y al leer se revisa antes de convertir.
ADO.NET	Librería de .NET para hablar con la base (`SqlConnection`, `SqlCommand`, `SqlDataAdapter`).
Cadena de conexión	Texto que dice a qué servidor y a qué base conectarse; vive solo en `Conexion`.
LocalDB	SQL Server liviano que se levanta bajo demanda, sin instalar un servidor.
Autenticación de Windows	`Integrated Security`: entra con el usuario de Windows y no hay contraseña de base en el código.
Pool de conexiones	Reserva de conexiones físicas que se reutilizan; por eso abrir y cerrar a cada consulta no es lento.

Término	En una frase
Parámetro	Hueco `@x` cuyo valor viaja aparte de la consulta; es lo que evita la inyección SQL.
Inyección SQL	Meter código SQL por un campo de texto; se evita con parámetros.
`ExecuteScalar`	Devuelve un solo valor (la primera columna de la primera fila).
`ExecuteNonQuery`	Ejecuta `INSERT/UPDATE/DELETE` y devuelve cuántas filas cambió.
`ExecuteReader`	Lee el resultado fila por fila.
`Fill` y `DataTable`	Trae todo el resultado a una tabla en memoria desconectada de la base.
`RowFilter`	Filtro en memoria sobre un `DataView`; usa un lenguaje de .NET, no SQL.
`SCOPE_IDENTITY()`	El último Id generado por esta ejecución; no se confunde con triggers como `@@IDENTITY`.
Transacción	Grupo de operaciones que se confirman todas (`Commit`) o se deshacen todas (`Rollback`).
Procedimiento almacenado	Programa guardado en la base que ejecuta una operación con sus reglas y su transacción.
Trigger	Código de la base que se dispara solo con un `INSERT`, `UPDATE` o `DELETE`; aquí, la auditoría.
`THROW` y `SqlException`	El error que lanza SQL con un número y un texto, y la excepción con que llega a C#.
`SESSION_CONTEXT`	Datos pegados a la conexión; así el trigger sabe qué usuario hizo el cambio.
Baja lógica	Cambiar el estado del registro en vez de borrarlo, para conservar su historial.
Hash, salt y BCrypt	Hash: función de un solo sentido; salt: valor aleatorio por contraseña; BCrypt: hash lento con salt y costo.
`RandomNumberGenerator`	Generador aleatorio criptográfico; se usa en vez de `Random` para claves y códigos.

Término	En una frase
`ErrorProvider` y `hayError`	Icono rojo junto al campo y la bandera que revisa todos los campos antes de avisar una sola vez.
`cargando`	Bandera que dice «estoy llenando la pantalla, no reacciones a los eventos».
GDI+, `Graphics` y `OnPaint`	La librería de dibujo, el lienzo y el método donde cada control se pinta.
`Invalidate()`	Pide a Windows que vuelva a llamar a `OnPaint`; el aspecto es función del estado.
Doble búfer	Armar el dibujo en memoria y mostrarlo de una vez para que no parpadee.
P/Invoke	Llamar funciones nativas de Windows (`SetWindowTheme`) desde C#.

11.2 Qué está corregido y qué sigue abierto

Dilo tú antes de que te lo pregunten: reconocer los límites con su solución es lo que más suena a ingeniero.

Ya corregido en la carpeta `entrega_final`:

- **Login.** Ahora autentica y autoriza siempre: la rama de claves antiguas en texto plano ya no se salta la comprobación de cuenta activa ni de rol, una clave vacía se rechaza y la migración a hash ocurre solo después de autorizar.
- **DUI.** El dígito verificador ya se calcula (se quitó el `return true;` que lo dejaba sin efecto).
- **Ventana.** El login y el panel arrancan maximizados, con botones de minimizar, maximizar y cerrar.

Sigue abierto, con su mejora:

- El modo Debug salta el login → presentar siempre en Release.
- La dirección y el token del servicio de correo están en el código fuente → regenerarlo y sacarlo a un archivo de configuración.
- Las reglas de contraseña difieren según la pantalla → una sola función de validación.
- Hay código duplicado (`CorreoValido` en cuatro formularios, `GenerarCodigo` en dos) y clases heredadas sin uso → utilidades compartidas y limpieza.
- La regla del director se basa en el texto del cargo y solo la impone la pantalla → identificador o columna propia, reforzado en la base.

- El bloqueo de intentos del login vive en memoria → guardarlo en la base.
- No hay concurrencia optimista (`rowversion`) ni pruebas automatizadas → agregarlas.
- La paleta de colores no está centralizada del todo → que todos lean de `PaletaSistema` .

11.3 Repaso final de 20 minutos

Hazlo en voz alta y con el cronómetro, sin mirar la guía; lo que te cueste, márcalo y vuelve a leerlo al final.

1. **3 min, los flujos.** Cuenta con la receta de cinco frases de la sección 9.4 el login, el préstamo y la planilla.
2. **4 min, SQL desde C#.** Escribe de memoria `YaExisteDirector` , explica cada línea y por qué `INNER JOIN` con cargos pero `LEFT JOIN` con planillas (P10 y P11).
3. **3 min, el viaje de un error.** De `THROW` en el procedimiento, pasando por `SqlException` y su `Number` , hasta el aviso con `ErrorProvider` o `MessageBox` (P18 y P19).
4. **4 min, seguridad.** Un minuto cada una: BCrypt contra cifrado (P26), el bloqueo y la recuperación con sus límites (P28), Debug contra Release y la inyección SQL con la demostración (P25 y P33).
5. **3 min, diseño.** El ciclo estado → `Invalidate()` → `OnPaint` , cómo se dibuja un icono y cómo se redondea una esquina (P2 y P30).
6. **3 min, límites y mejoras.** Di la lista de la sección 11.2 y las tres mejoras que harías primero (P38).

11.4 Antes de presentar: lista de comprobación

- Ejecutar y presentar la compilación **Release** y comprobar que pide usuario y contraseña.
- Probar la demostración de la inyección SQL en el login (`' OR '1'='1`) y ver que responde «Usuario o contraseña incorrectos».
- Ejecutar `SELECT Usuario FROM Usuarios WHERE Clave NOT LIKE '$2%'` y confirmar que no devuelve filas.
- Abrir tres pantallas (Personal, Préstamos y Planillas) y contar su flujo sin mirar.
- Tener abiertos en el editor `Datos.cs` , `frmLogin.cs` (`ValidarCredenciales`) y `frmAgregarPersonal.cs` (`YaExisteDirector` y la transacción), por si te piden ver el código.
- No mostrar ni leer en voz alta la dirección ni el token del servicio de correo.
- Si te preguntan algo que no sabes: reconocerlo con precisión, razonar con lo que sí sabes y abrir el código con el profesor (P40).